Missouri State University

Welcome to CSC 365

Internet Programming

Dr. Yifan Zhang

Learning Objectives

1. Understand the history and motivation for React
2. Be able to set up and build a basic React project
3. Differentiate between traditional web programming and React, including the use of JSX
4. Deconstruct websites into their basic components

What is React?

Definition:

- Also called ReactJS, React is a JS library for building user interfaces.
- Developed by Facebook, dating back to 2010.
- Started as an internal development tool, then open-sourced in 2013.

Why should we use React?

Among many reasons, it abstracts away interactions with the DOM and enables a more declarative way of constructing the user interface via components.

Similar Alternatives: Angular, Vue, Svelte

Traditional Component Construction

getElement, createElement, and appendChild

```
// Set the instructions
const instructionsHTML = document.getElementById("instructions");
for(let step of data.recipe) {
    const node = document.createElement("li");
    node.innerText = step;
    instructionsHTML.appendChild(node)
}
```

This gets quite long with many children! It also focuses on the how rather than the what.

Traditional Component Construction

innerHTML focuses on the what, but...

```
let html = `<div>`;
html += `<h2>${stud.name.first} ${stud.name.last}</h2>`;
html += `<p><strong>${stud.major}</strong></p>`;
html += `</div>`
document.getElementById('students').innerHTML = studentHtml;
```

...it is dangerous with untrusted input!

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites.

The Virtual DOM

Rather than updating the document ourselves, we describe to React what we want to display, and then React does the updating for us!

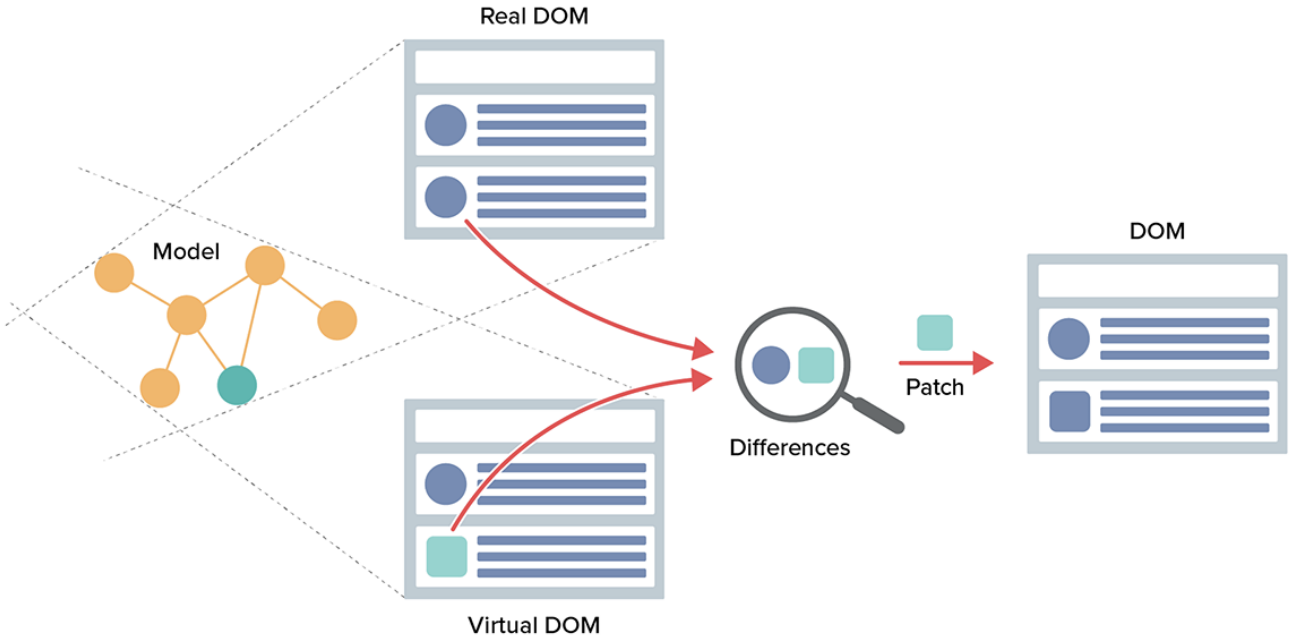
```
function Person(props) {
  return <div>
    <p>Hi! My name is {props.name} and I am {props.age} years old.</p>
  </div>
}
```

As name and age change, React will "react" and re-render the component with the new values.

Reconciliation

This is done through periodic reconciliation.

Reconciliation is the process of diffing and syncing the virtual and real DOM to render changes for the user.



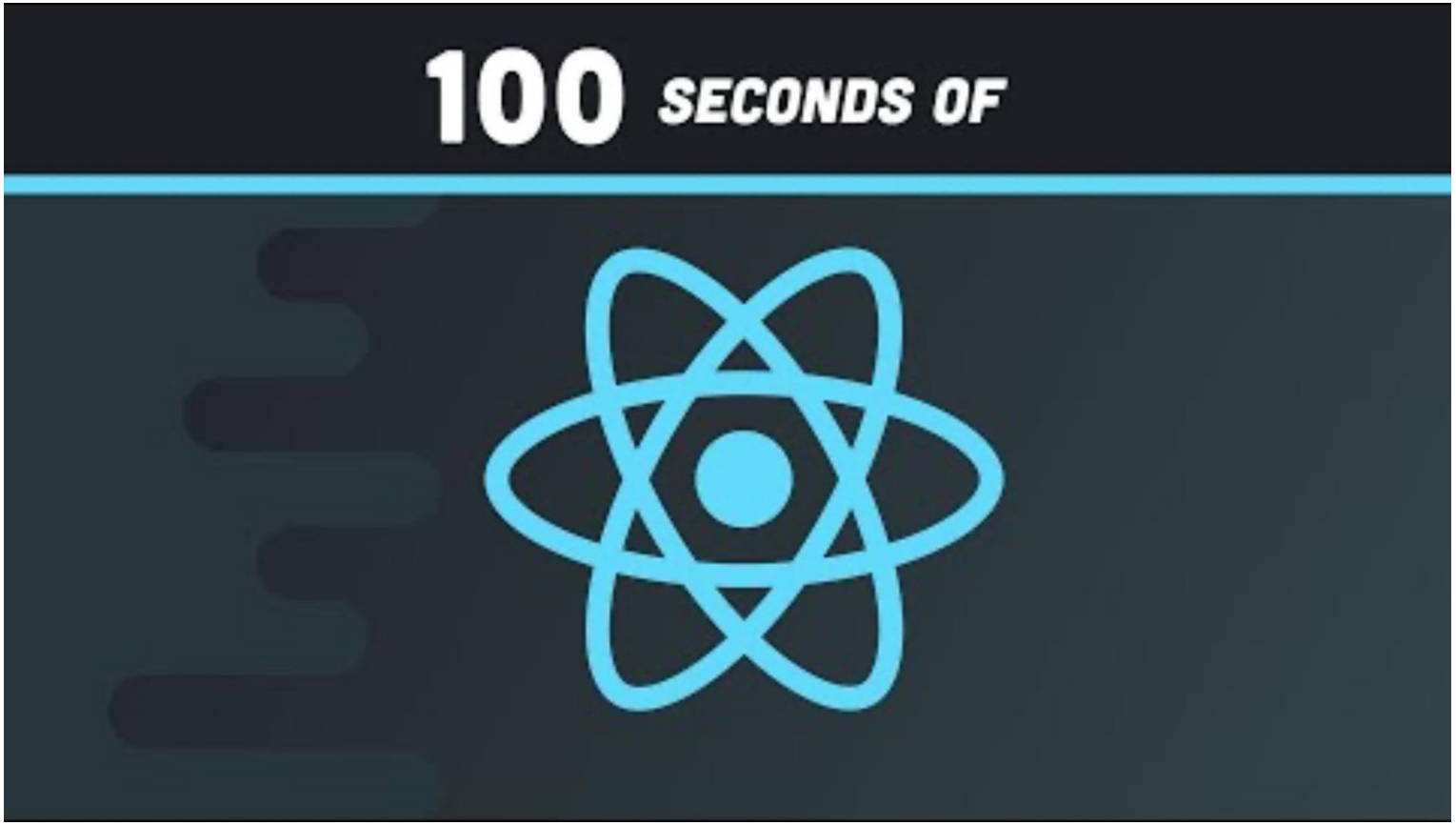
React DOM

react itself is platform-agnostic! It is simply an efficient tree reconciler

react-dom does the document transcription

by Meta

React in 100s



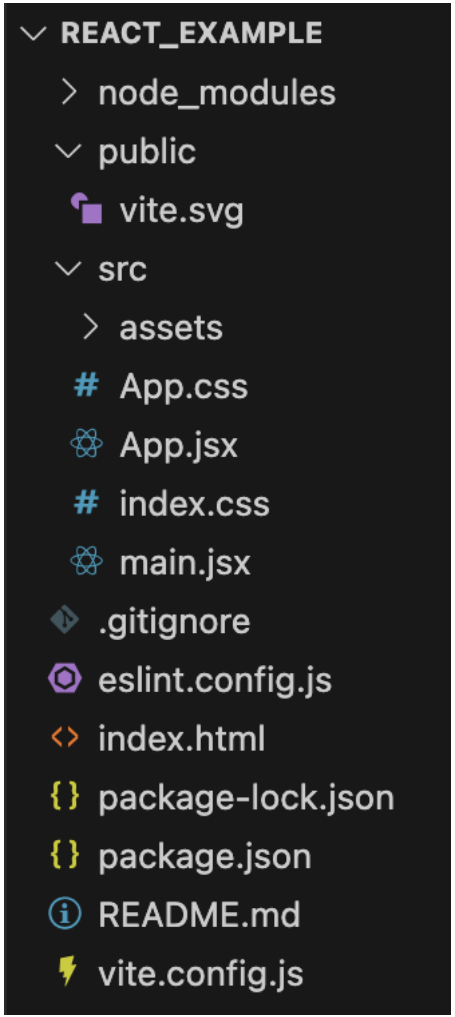
https://www.youtube.com/watch?v=Tn6-PIqc4UM

Getting Started

Install Node.js and NPM at:
https://nodejs.org/en/download

Run this command to create a React application named first_react:

```
npm create vite@latest first_react -- --template react
```

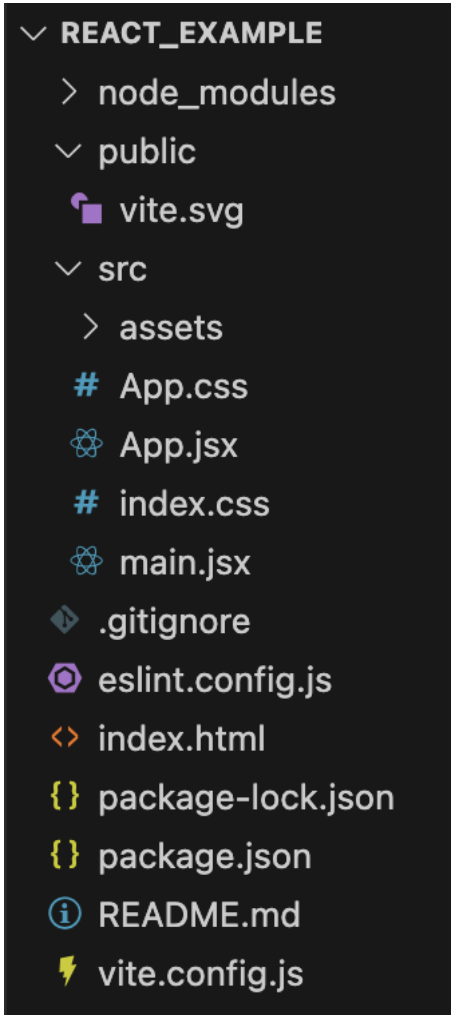


Getting Started

For an existing React project, you simply need to...

```
npm install
npm run dev
```

npm install downloads the necessary dependencies from npmjs.com specified in your package.json to a folder called node_modules
npm run dev starts a local webserver.
We don't open index.html anymore -- we go to localhost:5173



Hello World

Replace all the content of the ./src/App.jsx file with the code below:

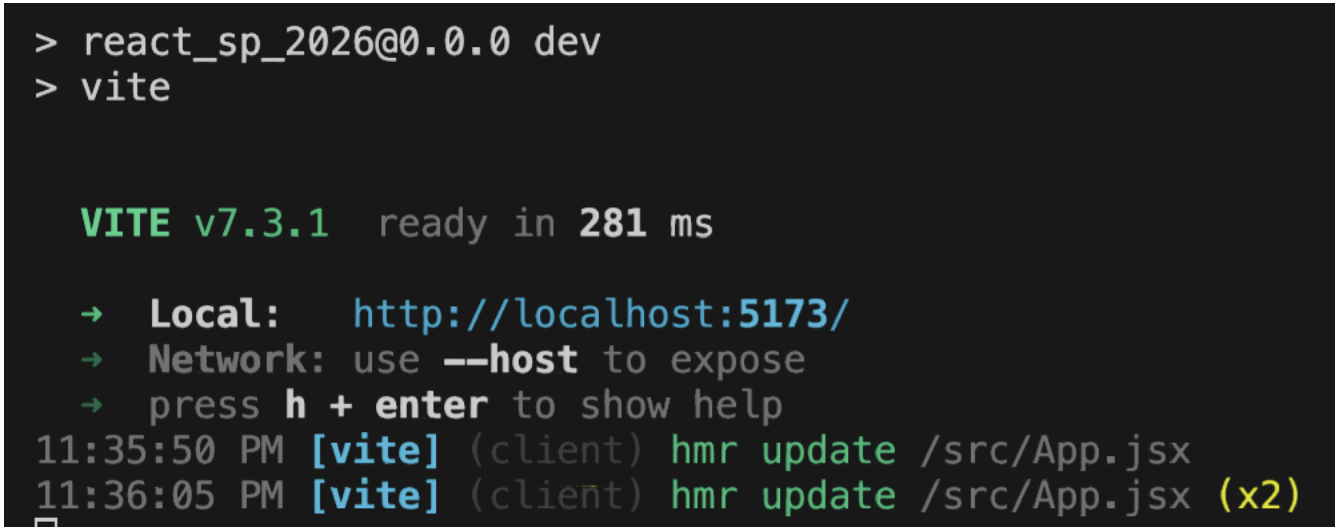
```
function App() {
  return (
    <div className="App">
      <h1>Hello World!</h1>
    </div>
  );
}

export default App;
```

Chaining Declarative Functions

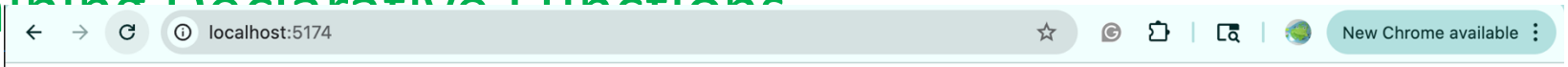
Copy and paste the localhost website to browser.

npm run dev starts a local webserver. We don't open index.html anymore -- we go to local address: http://localhost:5173/



Chaining Declarative Functions

Cop



Hello World!

React Component Basics

In React, every thing is a component. What is a thing?

- Similar question: what defines a class in Java?
- Some re-usable piece of the interface.
- May have many children, but only one parent.

Spotify Deconstruction

Spotify

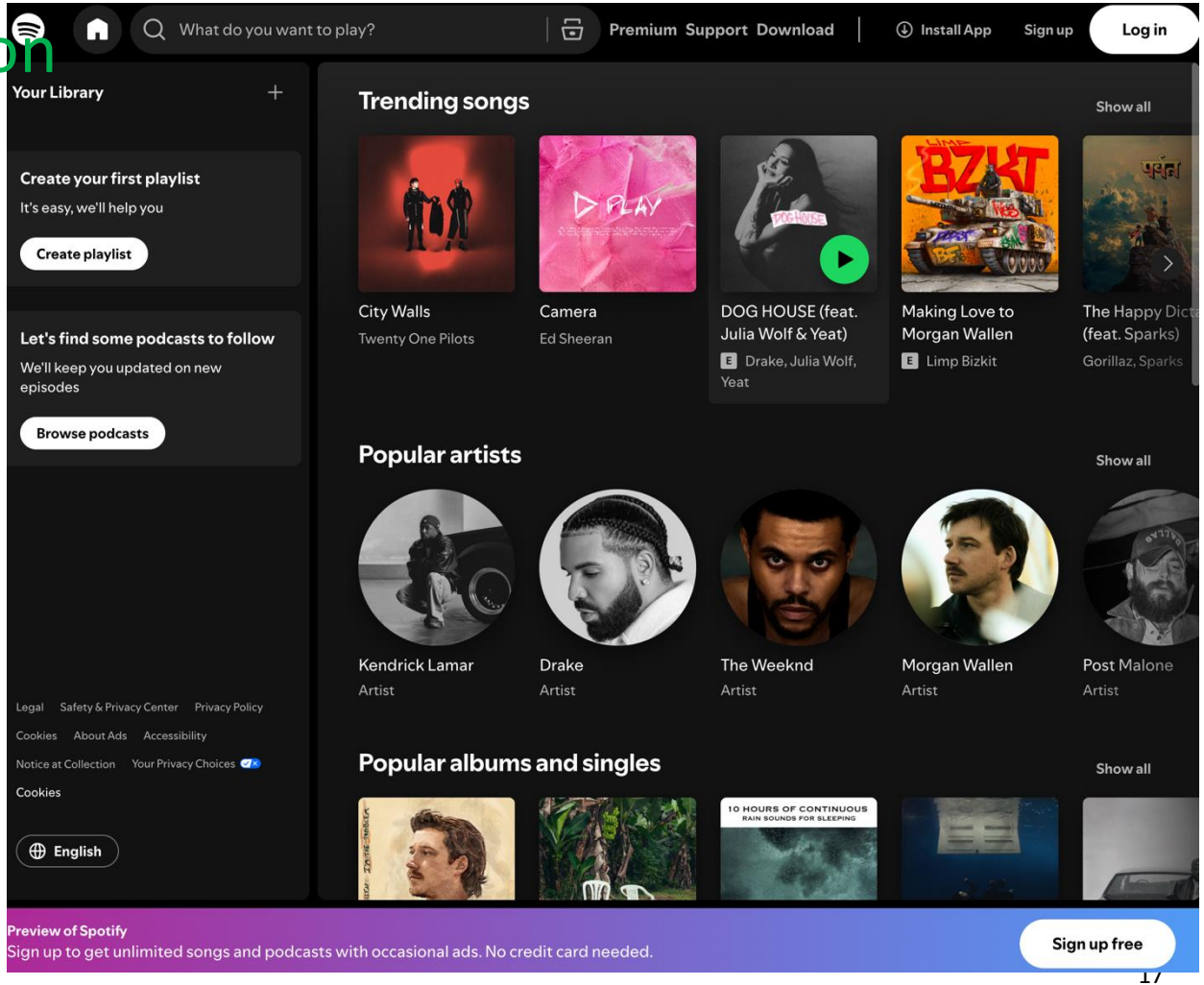
HeaderBar

MainContent

Trending songs: Card[]

Popular artists: Card[]

BottomBar: Button



Note: Class vs Functional Components

A class component looks like this...

```
class Welcome extends React.Component {
  render() {
    return <div>
      <h1>Hello World!</h1>
      <p>This is my website.</p>
    </div>
  }
}
```

Functional components were introduced in React 16.8.

React Component Basics

A functional component looks like this...

```
function Welcome() {
  return <div>
    <h1>Hello World!</h1>
    <p>This is my website.</p>
  </div>
}
```

It returns a single element of JSX. If you have more than one element, you can return a div or fragment <></>.

React Component Basics

JSX is a syntax that combines HTML, CSS, and JSX.

```
function Welcome(props) {
  return <div>
    <h1>Hello World!</h1>
    <p>{props.msg}</p>
  </div>
}
```

{} interpolates any JS expression and safely inserts it into the DOM. JSX is transpiled into HTML, CSS, and JS at runtime.

React Component Basics

For example, you can specify styling by interpolating a JavaScript object into your JSX.

```
function Welcome(props) {
  return (
    <div style={{ border: "dotted", padding: "1rem" }}>
      <h1>Hello World!</h1>
      <p>{props.msg}</p>
    </div>
  );
}
```

React Component Basics

This is good, but not reuseable

```
16  function Person() {
17    return (
18      <div style={{ border: "dotted", padding: "1rem" }}>
19        <h2>Yifan Zhang</h2>
20        <p>32 years old</p>
21        <p>Works as a(n) Instructor</p>
22      </div>
23    );
24  }
```

React Component Basics

This is better, reuseable

```
16  function Person({ name, age, job, style }) {
17    const box = { border: "dotted", padding: "1rem", ...style };
18    return (
19      <div style={box}>
20        <h2>{name}</h2>
21        <p>{age} years old</p>
22        <p>Works as a(n) {job}</p>
23      </div>
24    );
25  }
```

React Component Basics

This is better, reuseable

```
6   function App() {
7     return (
8       <div>
9         <Person name="Yifan Zhang" age={32} job="Instructor" />
10        <Person name="Stephen Curry" age={37} job="Basketball Player" style={{ border: "solid" }} />
11      </div>
12    );
13  }
```

Notice: props are passed down as key-value pairs.

Syntactic Sugar

Use the ... spread operator to pass down many props all at once.

Instead of writing: <Person name={TA.name} age={TA.age} />

We could use ... spread operator as follows:

```
const TA = {name: "Zach Potter", age: 24}

function YellowPages() {
  return <div>
    <Person {...TA}></Person>
  </div>
}
```

Practice

src > App.jsx > ...

```jsx
import './App.css'

function App() {
  return (
    <div>
      <Person name="Yifan Zhang" age={32} job="Instructor" />
      <Person name="Stephen Curry" age={37} job="Basketball Player" style={{ border: "solid" }} />
    </div>
  );
}

export default App

function Person({ name, age, job, style }) {
  const box = { border: "dotted", padding: "1rem", ...style };
  return (
    <div style={box}>
      <h2>{name}</h2>
      <p>{age} years old</p>
      <p>Works as a(n) {job}</p>
    </div>
  );
}
```

Syntactic Sugar

If the component does not take any children, you can even write it shorthand!

```
const TA = {name: "Zach Potter", age: 24}

function YellowPages() {
  return <div>
    <Person {...TA} />
  </div>
}
```

```
<Person {...TA} /> == <Person name={TA.name} age={TA.age} />
```

Syntactic Sugar

When a React component receives props, you can read them in two equivalent ways:

1. Access through the props object

```
function Person(props) {
  // use props.name, props.age, props.role, ...
  return <p>{props.name} is {props.age}</p>;
}
```

2. Destructure the props you need (syntactic sugar)

```
function Person({ name, age }) {
  // use name, age directly
  return <p>{name} is {age}</p>;
}
```

Design Sugar

React-Bootstrap is Bootstrap for React! You can import existing components such as Card, Button, and more! Beware of your imports!

```
import { Card, Button } from 'react-bootstrap';

export default function Person({ name, age }) {
  return (
    <Card>
      <p>Hi! My name is {name} and I am {age} years old.</p>
      <Button onClick={() => alert('hello!')}>Say hello!</Button>
    </Card>
  );
}
```

Imports and Exports

React-Bootstrap exports many components, we import them with curly braces, e.g.

```
import { Card, Button } from 'react-bootstrap'
```

We typically export one component by default (see last slide) and import directly as...

```
import Person from './person' // Use relative pathing for local files
```

Callback Handlers

In React, event listeners take callback functions as a parameter -- not function calls!

```
<Button onClick={() => alert('hello!')}>Say hello!</Button>
```

Correct!

```
<Button onClick={alert('hello!')}>Say hello!</Button>
```

Incorrect!

Reactive State Basics

We often use event listeners to mutate state. Whereas props is given by the parent component, state is managed internally.

```
const [name, setName] = useState("James");
```

We get back a read-only variable and a mutator function; do not mutate the variable directly!

Use the mutator function.

Reactive State Basics

Clicking the button calls the callback that uses setName, which updates the state and re-renders the UI.

```
import { useState } from "react";

export default function Hello() {
  const [name, setName] = useState("James");

  return (
    <div>
      <p>Hello, {name}!</p>
      <button onClick={() => setName("Taylor")}>Change name</button>
    </div>
  );
}
```

Reactive State Basics

State variables declared with useState are reactive, meaning that a change to its state (via its mutator function) will trigger a re-render!

```
setName("Taylor")
```

This re-render will happen in the near future, causing the function to be ran again.

Initial state is only set on component mount.

Reactive State Basics

```
function Person(props) {
  const [name, setName] = useState();     // declare a state variable

  function giveName() {
    const allNames = ["Amy","Ben","Eli","Ian","Leo","Mia"];
    const randI = Math.floor(Math.random() * allNames.length);
    setName(allNames[randI]);             // update state with a random name
  }

  return (
    <div>
      <h1>My name is {name}.</h1>
      <button onClick={giveName}>Give New Name</button>
    </div>
  );
}
```

Detour: Conditional Rendering

```
<h1>My name is {name}.</h1>
```

The initial render isn't ideal... How can we handle this?

```
{name ? <h1>My name is {name}.</h1> : <p>I don't have a name yet...</p>}
```

You control what is returned; do a conditional render!

Remember that {} will interpolate any JS expression.

Reactive State Basics

```
function Person(props) {
  const [name, setName] = useState();

  function giveName() {
    const allNames = ["Amy", "Ben", "Eli", "Ian", "Leo", "Mia"];
    const randI = Math.floor(Math.random() * allNames.length);
    setName(allNames[randI]);
  }

  return <div>
    {name ? <h1>My name is {name}.</h1> : <p>I don't have a name yet...</p>}
    <button onClick={giveName}>Give New Name</button>
  </div>
}
```

Reactive State Basics

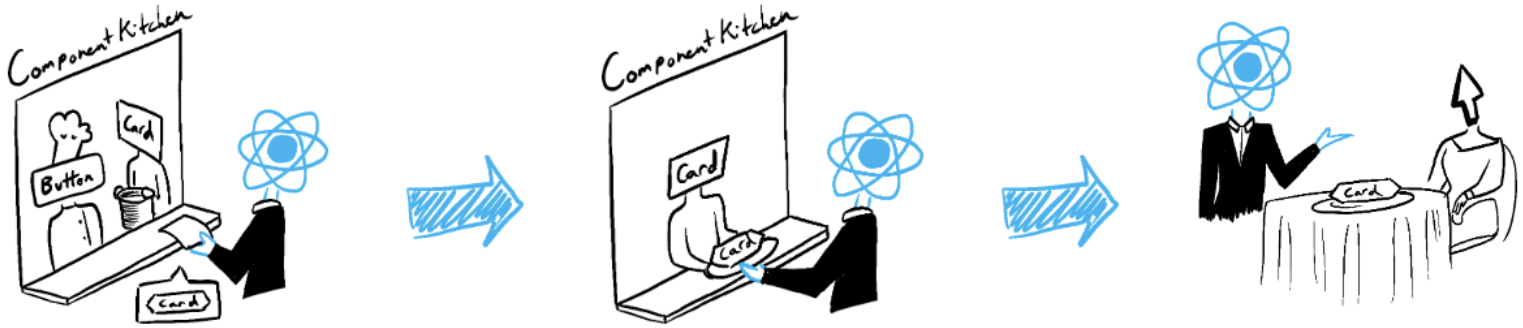
Finally, add a console.log('My name is', name) immediately after calling setName and click the button again... What do you notice?

```
My name is undefined
```

It is its previous value, why do you think this is?

Reactive State Basics

Just because the food is cooked doesn‘t mean the server has already brought it to the table!



Trigger Render Commit

2 Ways to Set State

```
setLikes(0)
```

overwrites the existing value

```
setLikes(old => old + 1)
```

updates existing value

The second notation (using a callback function) is never a wrong answer, it can just be verbose. What you return is what the state will be set as.

ICA 9

Person.jsx × main.jsx

src > Person.jsx > Person

```jsx
import React, { useState } from 'react'

export default function Person(props) {
  const [name, setName] = useState();

  function giveName() {
    const allNames = ["Amy", "Ben", "Eli", "Ian", "Leo", "Mia"];
    const randI = Math.floor(Math.random() * allNames.length);
    setName(allNames[randI]);
  }

  return <div>
    {name
        ? <h1>My name is {name}.</h1>   // 3) conditional render when we have a name
        : <p>I don't have a name yet...</p>}
    <button onClick={giveName}>Give New Name</button>
  </div>
}
```

ICA 9

Person.jsx | main.jsx

src > main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
// import App from './App.jsx'
import Person from './Person.jsx'

createRoot(document.getElementById('root')).render(
  // <StrictMode>
  //   <App />
  // </StrictMode>,
  <Person />
)
```